MATERIAL DIDÁCTICO · ACCESO A DATOS

TUTORIAL RESUELTO 01 · UNIDAD 02

JDBC desde cero con MySQL

Instalación local, proyecto Maven, conexión segura, operaciones CRUD y verificación paso a paso.

Objetivo y resultado

Construir un CRUD real en Java con una base MySQL, sin instalar MySQL manualmente y sin escribir dependencias a mano. Al terminar podrás listar clubes, crear uno, buscarlo, cambiar su ciudad y borrarlo; verás esos cambios en la base de datos.

Requisitos
JDK 17 o superior, Maven, Docker Desktop (Windows) o Docker Engine con Compose, y un editor Java. La primera ejecución requiere conexión a Internet para descargar la imagen del servidor y las dependencias Maven.

Paso 1. Comprobar herramientas

Abre una terminal (PowerShell en Windows) y ejecuta cada comando. Deben mostrar versiones sin errores.

TERMINAL

```
java -version
mvn -version
docker --version
docker compose version
```

Paso 2. Encender el gestor y verificar el esquema

Sitúate en la carpeta **Codigo/Infraestructura/mysql** y ejecuta lo siguiente. La descarga inicial puede tardar; para seguir, comprueba que el contenedor aparece como sano (healthy).

TERMINAL

```
docker compose up -d
docker compose ps
docker compose exec mysql mysql -uad_user -p ad_ud2
```

Cuando MySQL pida contraseña escribe **ad_pass**. Dentro de la consola SQL introduce estas instrucciones (el último punto y coma es necesario):

SQL

```
SHOW TABLES;
SELECT id, nombre, presupuesto FROM club ORDER BY id;
SELECT id, nombre, club_id FROM jugador ORDER BY id;
EXIT;
```

Deberían aparecer tres tablas (club, jugador, movimiento) y tres clubes: Granada CF, Málaga CF y Cádiz CF. Si la tabla no aparece, revisa los logs con **docker compose logs mysql**.

Paso 3. Examinar la infraestructura

El archivo compose define una instancia local y el SQL configura el esquema. No hace falta crear las tablas a mano; ambos ficheros se incluyen completos a continuación.

3.1. Configuración de MySQL

Ruta: **Codigo/Infraestructura/mysql/compose.yaml**

ARCHIVO COMPLETO

```
services:
  mysql:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: root_clase
      MYSQL_DATABASE: ad_ud2
      MYSQL_USER: ad_user
      MYSQL_PASSWORD: ad_pass
    ports:
      - "127.0.0.1:3307:3306"
    volumes:
      - mysql_ad_ud2:/var/lib/mysql
      - ./init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -u root -p$$MYSQL_ROOT_PASSWORD --silent"]
      interval: 5s
      timeout: 5s
      retries: 24
volumes:
  mysql_ad_ud2:
```

3.2. Estructura y datos iniciales

Ruta: **Codigo/Infraestructura/mysql/init/01_esquema.sql**

ARCHIVO COMPLETO

```
USE ad_ud2;

CREATE TABLE IF NOT EXISTS club (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(90) NOT NULL UNIQUE,
  ciudad VARCHAR(90) NOT NULL,
  presupuesto DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  CONSTRAINT chk_presupuesto CHECK (presupuesto >= 0)
);

CREATE TABLE IF NOT EXISTS jugador (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(90) NOT NULL,
  dorsal INT NOT NULL,
  club_id INT NOT NULL,
  CONSTRAINT fk_jugador_club FOREIGN KEY (club_id)
    REFERENCES club(id) ON DELETE RESTRICT,
  CONSTRAINT uq_dorsal_club UNIQUE (club_id, dorsal)
);

CREATE TABLE IF NOT EXISTS movimiento (
```

```
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  club_origen_id INT NOT NULL,
  club_destino_id INT NOT NULL,
  importe DECIMAL(10,2) NOT NULL,
  fecha TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_importe CHECK (importe > 0),
  CONSTRAINT fk_mov_origen FOREIGN KEY (club_origen_id) REFERENCES club(id),
  CONSTRAINT fk_mov_destino FOREIGN KEY (club_destino_id) REFERENCES club(id)
);

INSERT INTO club (id, nombre, ciudad, presupuesto) VALUES
  (1, 'Granada CF', 'Granada', 2500.00),
  (2, 'Málaga CF', 'Málaga', 1500.00),
  (3, 'Cádiz CF', 'Cádiz', 1000.00)
ON DUPLICATE KEY UPDATE id = id;

INSERT INTO jugador (id, nombre, dorsal, club_id) VALUES
  (1, 'Lucía', 10, 1),
  (2, 'Pablo', 7, 1),
  (3, 'Sara', 9, 2)
ON DUPLICATE KEY UPDATE id = id;

DROP PROCEDURE IF EXISTS contar_jugadores;
DELIMITER //
CREATE PROCEDURE contar_jugadores(IN p_club_id INT, OUT p_total INT)
BEGIN
  SELECT COUNT(*) INTO p_total
  FROM jugador WHERE club_id = p_club_id;
END //
DELIMITER ;
```

El procedimiento contar_jugadores se utilizará en el segundo tutorial. DELIMITER es una instrucción del cliente MySQL para permitir puntos y coma dentro del cuerpo del procedimiento.

Paso 4. Abrir el proyecto Java

En IntelliJ IDEA: **File → Open** y selecciona la carpeta Codigo/T01_JDBC_CRUD (o su pom.xml). En VS Code, abre esa carpeta y deja que la extensión Java cargue el proyecto. El código también puede ejecutarse desde la terminal sin IDE.

TERMINAL

```
cd Codigo/T01_JDBC_CRUD
mvn clean compile exec:java
```

Maven descarga mysql-connector-j y compila las clases. El proyecto no requiere descargar un JAR de forma manual. Si Maven no aparece en PowerShell, instala Maven y añade su carpeta bin a PATH.

Paso 5. Entender los archivos

El proyecto contiene cuatro archivos: **pom.xml** declara dependencias, **Conexion.java** abre conexiones, **Club.java** define los datos que devuelve el DAO, **ClubDao.java** implementa el CRUD y **Main.java** controla la salida.

5.1. Dependencias Maven

Ruta: **Codigo/T01_JDBC_CRUD/pom.xml**

ARCHIVO COMPLETO

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.curso.ad</groupId>
  <artifactId>t01-jdbc-crud</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>8.4.0</version>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.5.0</version>
        <configuration><mainClass>es.curso.ad.t01.Main</mainClass></configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

5.2. Clase Conexion

Ruta: **Codigo/T01_JDBC_CRUD/src/main/java/es/curso/ad/t01/Conexion.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t01;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public final class Conexion {
    private static final String URL =
            "jdbc:mysql://localhost:3307/ad_ud2?connectionTimeZone=UTC";
    private static final String USUARIO = "ad_user";
    private static final String CLAVE = "ad_pass";

    private Conexion() { }

    public static Connection abrir() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, CLAVE);
    }
```

```
}
```

El método abrir devuelve Connection o lanza SQLException; no imprime nada. La URL usa el puerto externo 3307 y el esquema ad_ud2.

5.3. Registro de datos Club

Ruta: **Codigo/T01_JDBC_CRUD/src/main/java/es/curso/ad/t01/Club.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t01;

import java.math.BigDecimal;

public record Club(int id, String nombre, String ciudad,
                   BigDecimal presupuesto) { }
```

BigDecimal conserva cantidades decimales como 120.50 sin pasar por la representación binaria de double.

5.4. DAO con todas las operaciones

Ruta: **Codigo/T01_JDBC_CRUD/src/main/java/es/curso/ad/t01/ClubDao.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t01;

import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class ClubDao {
    public int insertar(String nombre, String ciudad,
                        BigDecimal presupuesto) throws SQLException {
        String sql = "INSERT INTO club(nombre, ciudad, presupuesto) VALUES (?, ?, ?)";
        try (Connection conexion = Conexion.abrir();
             PreparedStatement sentencia = conexion.prepareStatement(
                     sql, Statement.RETURN_GENERATED_KEYS)) {
            sentencia.setString(1, nombre);
            sentencia.setString(2, ciudad);
            sentencia.setBigDecimal(3, presupuesto);
            sentencia.executeUpdate();
            try (ResultSet claves = sentencia.getGeneratedKeys()) {
                if (claves.next()) {
                    return claves.getInt(1);
                }
                throw new SQLException("No se ha generado el ID del club");
            }
        }
    }
```

```
    public List<Club> listar() throws SQLException {
        String sql = "SELECT id, nombre, ciudad, presupuesto FROM club ORDER BY id";
        List<Club> clubes = new ArrayList<>();
        try (Connection conexion = Conexion.abrir();
             PreparedStatement sentencia = conexion.prepareStatement(sql);
             ResultSet filas = sentencia.executeQuery()) {
            while (filas.next()) {
                clubes.add(leerClub(filas));
            }
        }
        return clubes;
    }

    public Optional<Club> buscar(int id) throws SQLException {
        String sql = "SELECT id, nombre, ciudad, presupuesto FROM club WHERE id = ?";
        try (Connection conexion = Conexion.abrir();
             PreparedStatement sentencia = conexion.prepareStatement(sql)) {
            sentencia.setInt(1, id);
            try (ResultSet filas = sentencia.executeQuery()) {
                if (filas.next()) {
                    return Optional.of(leerClub(filas));
                }
            }
        }
        return Optional.empty();
    }

    public boolean cambiarCiudad(int id, String ciudad) throws SQLException {
        String sql = "UPDATE club SET ciudad = ? WHERE id = ?";
        try (Connection conexion = Conexion.abrir();
             PreparedStatement sentencia = conexion.prepareStatement(sql)) {
            sentencia.setString(1, ciudad);
            sentencia.setInt(2, id);
            return sentencia.executeUpdate() == 1;
        }
    }

    public boolean borrar(int id) throws SQLException {
        String sql = "DELETE FROM club WHERE id = ?";
        try (Connection conexion = Conexion.abrir();
             PreparedStatement sentencia = conexion.prepareStatement(sql)) {
            sentencia.setInt(1, id);
            return sentencia.executeUpdate() == 1;
        }
    }

    private Club leerClub(ResultSet filas) throws SQLException {
        return new Club(
                filas.getInt("id"),
                filas.getString("nombre"),
                filas.getString("ciudad"),
                filas.getBigDecimal("presupuesto"));
    }
}
```

Cada método abre y cierra sus propios recursos. Los parámetros SQL se representan mediante ?, y los métodos devuelven objetos o resultados de operación.

5.5. Programa principal

Ruta: **Codigo/T01_JDBC_CRUD/src/main/java/es/curso/ad/t01/Main.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t01;

import java.math.BigDecimal;
import java.sql.SQLException;

public class Main {
    public static void main(String[] args) {
        ClubDao dao = new ClubDao();
        try {
            System.out.println("CLUBES INICIALES");
            for (Club club : dao.listar()) {
                System.out.println(club);
            }

            String nombreTemporal = "Club de prueba JDBC";
            int id = dao.insertar(nombreTemporal, "Granada", new BigDecimal("120.50"));
            System.out.println("Creado con ID " + id);
            System.out.println("Buscado: " + dao.buscar(id).orElseThrow());
            System.out.println("Actualizado: " + dao.cambiarCiudad(id, "Jaén"));
            System.out.println("Tras cambiar: " + dao.buscar(id).orElseThrow());
            System.out.println("Borrado: " + dao.borrar(id));
            System.out.println("¿Sigue existiendo? " + dao.buscar(id).isPresent());
        } catch (SQLException e) {
            System.err.println("Error de base de datos: " + e.getMessage());
        }
    }
}
```

Main es el único lugar en el que mostramos datos. Inserta un club temporal, lo modifica y luego lo elimina; por eso pueden repetirse las ejecuciones sin acumular clubes de prueba.

Paso 6. Comprobar la salida y los errores

Al ejecutar la aplicación deben listarse al menos los tres clubes iniciales. Después verás un identificador nuevo, una búsqueda correcta, una actualización con true, un borrado con true y una comprobación final con false.

Ejecuta de nuevo y observa que los clubes iniciales siguen ahí. A continuación prueba a parar solo el contenedor con **docker compose stop mysql** desde la carpeta de infraestructura; al repetir Maven debe mostrarse un error SQL. Recupera el servidor con **docker compose start mysql**.

Paso 7. Comprobaciones antes de R01

- Localiza la base de datos desde el cliente MySQL y verifica que no se ha guardado el club temporal.
- Explica para qué sirven executeQuery, executeUpdate y getGeneratedKeys.
- Confirma por qué setInt(1, id) utiliza 1 y no 0 como índice.

- Modifica temporalmente el nombre de prueba para que coincida con uno ya existente y observa cómo informa la restricción UNIQUE; luego restaura el código.

Paso 8. Comparar con una base embebida H2

El currículo también contempla gestores embebidos. En este segundo ejemplo JDBC no necesitas Docker: el motor H2 se incorpora como una dependencia Maven y almacena el esquema en un fichero local .mv.db. Las clases Connection, PreparedStatement y ResultSet son las mismas.

TERMINAL

```
cd Codigo/T01B_H2_Embebida
mvn clean compile exec:java
```

Abre el proyecto T01B_H2_Embebida. Ejecuta el programa dos veces y comprueba que en datos/clubes_h2.mv.db se conserva un único club. La sentencia MERGE actualiza o inserta por ID, en lugar de acumular duplicados. Esta es una diferencia del ejemplo, no una propiedad universal de bases embebidas.

8.1. Dependencia Maven de H2

Ruta: **Codigo/T01B_H2_Embebida/pom.xml**

ARCHIVO COMPLETO

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.curso.ad</groupId>
  <artifactId>t01b-h2-embebida</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <version>2.3.232</version>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.5.0</version>
        <configuration><mainClass>es.curso.ad.h2.Main</mainClass></configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

8.2. Ejemplo H2 embebido completo

Ruta: **Codigo/T01B_H2_Embebida/src/main/java/es/curso/ad/h2/Main.java**

ARCHIVO COMPLETO

```
package es.curso.ad.h2;

import java.nio.file.Files;
import java.nio.file.Path;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class Main {
    public static void main(String[] args) {
        Path carpeta = Path.of("datos");
        try {
            Files.createDirectories(carpeta);
            String url = "jdbc:h2:file:./datos/clubes_h2";
            try (Connection conexion = DriverManager.getConnection(url, "sa", "")) {
                try (Statement ddl = conexion.createStatement()) {
                    ddl.executeUpdate("CREATE TABLE IF NOT EXISTS club (" +
                            "id INT PRIMARY KEY, " +
                            "nombre VARCHAR(90) NOT NULL, " +
                            "ciudad VARCHAR(90) NOT NULL)");
                }
                try (PreparedStatement alta = conexion.prepareStatement(
                        "MERGE INTO club KEY(id) VALUES (?, ?, ?)")) {
                    alta.setInt(1, 1);
                    alta.setString(2, "Granada CF");
                    alta.setString(3, "Granada");
                    alta.executeUpdate();
                }
                try (PreparedStatement consulta = conexion.prepareStatement(
                        "SELECT id, nombre, ciudad FROM club ORDER BY id");
                     ResultSet filas = consulta.executeQuery()) {
                    while (filas.next()) {
                        System.out.println(filas.getInt("id") + " | " +
                                filas.getString("nombre") + " | " +
                                filas.getString("ciudad"));
                    }
                }
            }
        } catch (Exception e) {
            System.err.println("No se pudo operar con H2: " + e.getMessage());
        }
    }
}
```

H2 requiere un driver distinto, pero JDBC conserva los métodos principales. Si borras el fichero de base de datos tras cerrar el programa, se perderán los datos; es diferente de detener el proceso Java.

Paso 9. Puente con la UD1: migrar un fichero TXT

También se incluye la clase ImportarTxt.java en T01_JDBC_CRUD y un MainImportacion independiente. Valida todas las líneas primero; después inserta los registros dentro de una transacción. Consulta el apartado 13 del manual principal y prueba una importación exitosa y la repetición fallida por IDs duplicados.

TERMINAL

```
cd Codigo/T01_JDBC_CRUD
mvn compile exec:java -Dexec.mainClass=es.curso.ad.t01.MainImportacion
```

9.1. Código completo de la migración TXT a MySQL

Ruta: **Codigo/T01_JDBC_CRUD/src/main/java/es/curso/ad/t01/ImportarTxt.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t01;

import java.io.IOException;
import java.math.BigDecimal;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/** Importación completa: primero valida el TXT y después hace INSERT transaccional. */
public class ImportarTxt {
    public record ClubTxt(int id, String nombre, String ciudad) { }

    public List<ClubTxt> validar(Path archivo) throws IOException {
        List<ClubTxt> clubes = new ArrayList<>();
        Set<Integer> ids = new HashSet<>();
        for (String linea : Files.readAllLines(archivo, StandardCharsets.UTF_8)) {
            if (linea.isBlank()) continue;
            String[] campos = linea.split(";", -1);
            if (campos.length != 3 || campos[1].isBlank() || campos[2].isBlank()) {
                throw new IllegalArgumentException("Línea incorrecta: " + linea);
            }
            int id;
            try {
                id = Integer.parseInt(campos[0]);
            } catch (NumberFormatException e) {
                throw new IllegalArgumentException("ID incorrecto: " + linea, e);
            }
            if (id <= 0 || !ids.add(id)) {
                throw new IllegalArgumentException("ID duplicado/no válido: " + id);
            }
            clubes.add(new ClubTxt(id, campos[1].trim(), campos[2].trim()));
        }
        return clubes;
    }
```

```
    public int importar(Path archivo) throws IOException, SQLException {
        List<ClubTxt> validados = validar(archivo);
        try (Connection conexion = Conexion.abrir()) {
            conexion.setAutoCommit(false);
            try (PreparedStatement sentencia = conexion.prepareStatement(
                    "INSERT INTO club(id, nombre, ciudad, presupuesto) " +
                    "VALUES (?, ?, ?, ?)")) {
                for (ClubTxt club : validados) {
                    sentencia.setInt(1, club.id());
                    sentencia.setString(2, club.nombre());
                    sentencia.setString(3, club.ciudad());
                    sentencia.setBigDecimal(4, BigDecimal.ZERO);
                    sentencia.executeUpdate();
                }
                conexion.commit();
                return validados.size();
            } catch (SQLException | RuntimeException e) {
                try {
                    conexion.rollback();
                } catch (SQLException rollbackError) {
                    e.addSuppressed(rollbackError);
                }
                throw e;
            }
        }
    }
}
```

El fichero se valida entero antes de ejecutar INSERT. En caso de error SQL, la transacción revierte. Ningún método de persistencia imprime.

9.2. Main de la importación

Ruta: **Codigo/T01_JDBC_CRUD/src/main/java/es/curso/ad/t01/MainImportacion.java**

ARCHIVO COMPLETO

```
package es.curso.ad.t01;

import java.io.IOException;
import java.nio.file.Path;
import java.sql.SQLException;

public class MainImportacion {
    public static void main(String[] args) {
        try {
            int importados = new ImportarTxt().importar(
                    Path.of("datos", "clubes_importacion.txt"));
            System.out.println("Clubes importados: " + importados);
        } catch (IOException | SQLException | IllegalArgumentException e) {
            System.err.println("No se pudo importar: " + e.getMessage());
        }
    }
}
```